
UML DOCUMENTATION

BCanteen Digital Foodcourt Ordering System

1. Introduction

BCanteen is a digital foodcourt ordering system designed to allow users to browse foodcourts, explore tenant menus, and place orders without the need for an account or login. The system follows a client-server architecture where a Vue.js frontend communicates with an Express.js REST API backed by a MySQL relational database.

This document presents four UML diagrams that describe the system from different perspectives:

- Use Case Diagram – describes the interactions between the User actor and the system.
- Class Diagram – defines the domain entities, their attributes, and relationships.
- Sequence Diagram – illustrates the runtime message flow for two critical operations.
- Entity Relationship Diagram (ERD) – models the database schema and table relationships.

1.1 Technology Stack

Layer	Technology
Frontend	Vue 3 (Composition API), Pinia store, Vue Router, Axios
Backend	Node.js, Express.js, REST API
Database	MySQL 8 (relational)
API Response	Wrapped: { success: true, data: <payload> }
Auth	None – no login required
Cart	Pinia store (in-memory, single-tenant constraint)

1.2 User Flow

The primary user flow in BCanteen is a linear sequence:

- Landing Page → Home Page (browse all foodcourts)
- Home Page → Foodcourt Detail Page (view tenants in a foodcourt)
- Foodcourt Detail → Tenant Menu Page (view menu items)
- Tenant Menu → Cart Page (review selected items)
- Cart Page → Checkout Page (enter notes, select payment method)
- Checkout → Order Success Page (display generated order code)

2. Use Case Diagram

2.1 Overview

A Use Case Diagram shows the functional requirements of a system from the perspective of its users (actors). It identifies what the system does without specifying how it does it. In BCanteen, there is a single primary actor: the User.

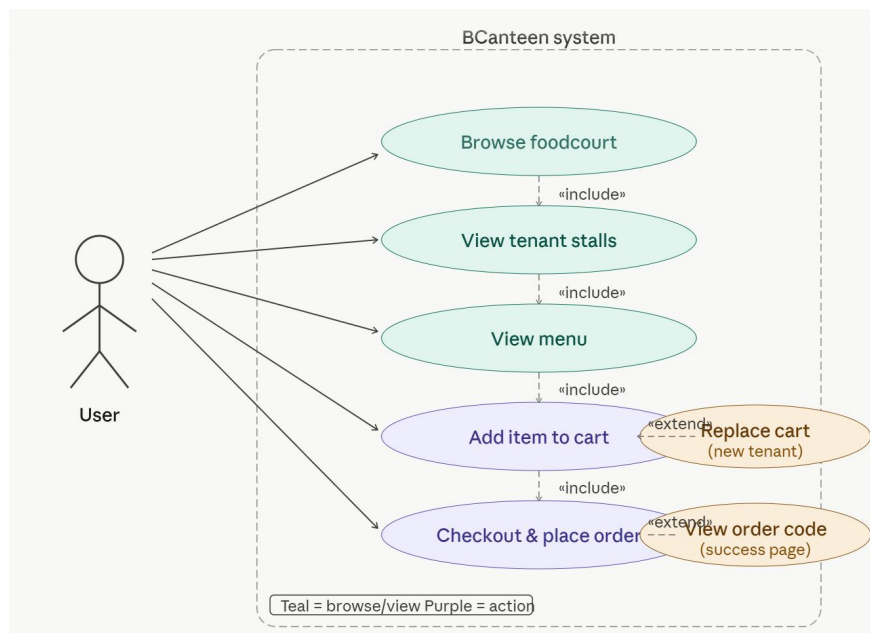


Figure 2.1 – BCanteen Use Case Diagram

2.2 Actor

Actor	Description
User	Any person who accesses the BCanteen web application from a browser or mobile device. No login or registration is required. The User can browse, order, and track their order using only an anonymous session.

2.3 Use Cases

The following use cases are identified within the BCanteen system boundary:

Use Case	Description
Browse foodcourt	The User can view a list of all available foodcourts fetched from GET /foodcourts. Each foodcourt card displays the name, image, rating, distance, and cuisine tags.
View tenant stalls	After selecting a foodcourt, the User can see all tenants (stalls) belonging to that foodcourt via GET /foodcourts/:id. Tenants are filterable by cuisine type.
View menu	After selecting a tenant, the User can view the full menu via GET /tenants/:id/menus. Items are grouped by category (Noodles, Rice,

	Drinks, etc.).
Add item to cart	The User can add available menu items to the cart. The cart is managed by the Pinia CartStore. A constraint enforces that only items from a single tenant may be in the cart at one time.
Checkout & place order	The User reviews the cart, enters optional notes, selects a payment method (QRIS or M-Banking), and submits the order via POST /orders. The system returns a unique order_code.
Replace cart	<<extend>> If the User attempts to add an item from a different tenant than the current cart, a confirmation dialog appears offering to clear the current cart and start fresh.
View order code	<<extend>> Upon successful order placement, the User is redirected to the Order Success page where the generated order_code is displayed prominently.

3. Class Diagram

3.1 Overview

The Class Diagram defines the domain model of BCanteen by identifying the key entities (classes), their attributes with data types, their methods, and the associations between them. It bridges the gap between the conceptual data model (ERD) and the runtime behaviour (Sequence Diagram).

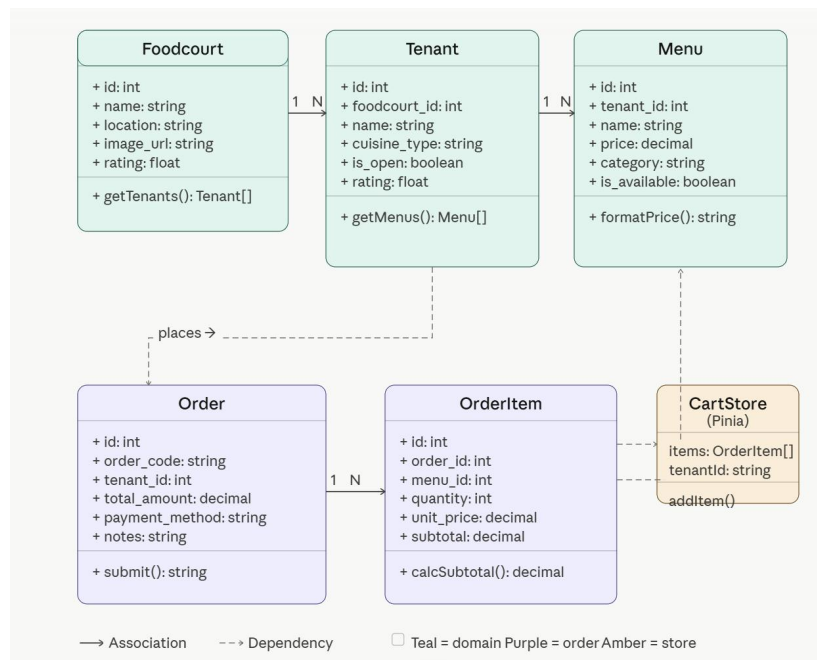


Figure 3.1 – BCanteen Class Diagram

3.2 Classes

Five domain classes are identified:

Foodcourt

Represents a physical foodcourt location. It is the top-level domain object.

Attribute / Method	Type / Return
id	int (PK)
name	string
location	string
image_url	string
rating	float
review_count	int
distance_km	float
peak_hours_start / end	string (time)
directions	string
created_at	timestamp
getTenants()	Tenant[]

Tenant

Represents a food stall or restaurant within a foodcourt. Each Tenant belongs to exactly one Foodcourt.

Attribute / Method	Type / Return
id	int (PK)
foodcourt_id	int (FK → Foodcourt)
name	string
cuisine_type	string
image_url	string
rating	float
is_open	boolean
prep_time_min	int
description	string
getMenus()	Menu[]

Menu

Represents a single orderable item offered by a Tenant.

Attribute / Method	Type / Return
id	int (PK)
tenant_id	int (FK → Tenant)
name	string
description	string
price	decimal
category	string
image_url	string
is_available	boolean
formatPrice()	string (e.g. "Rp 18.000")

Order

Represents a confirmed customer order created when the User submits checkout.

Attribute / Method	Type / Return
id	int (PK)
order_code	string (e.g. ORD-7F2A)
tenant_id	int (FK → Tenant)
total_amount	decimal
tax_amount	decimal
payment_method	string (QRIS M-Banking)
status	enum (pending confirmed ready)
notes	text
created_at	timestamp
submit()	string (returns order_code)

OrderItem

A line item within an Order. Stores a snapshot of the price at the time of ordering, so historical orders remain accurate even if menu prices change later.

Attribute / Method	Type / Return
id	int (PK)

order_id	int (FK → Order)
menu_id	int (FK → Menu)
quantity	int
unit_price	decimal (snapshot)
subtotal	decimal
calcSubtotal()	decimal (quantity × unit_price)

CartStore (Pinia)

Not a database entity, but a runtime class managed by the Pinia state management library on the frontend. It holds the active cart in memory and enforces the single-tenant constraint.

Attribute / Method	Description
items	OrderItem[] – current cart contents
tenantId	string – the tenant all items must belong to
tenantName	string – displayed in the replace-cart dialog
totalItems	computed: total quantity across all items
totalPrice	computed: sum of all item subtotals
addItem(item)	Adds item, enforces single-tenant rule
removeItem(id)	Decrements quantity or removes item
clearCart()	Empties cart and resets tenantId
getItemQty(id)	Returns current quantity for a menu item

3.3 Associations

Relationship	Cardinality	Description
Foodcourt → Tenant	1 : N	One foodcourt contains many tenant stalls.
Tenant → Menu	1 : N	One tenant offers many menu items.
Tenant → Order	1 : N	One tenant receives many orders (all items must be from the same tenant per order).
Order → OrderItem	1 : N	One order contains one or more line items.

Menu → OrderItem	1 : N	One menu item may appear in many order items across different orders.
CartStore → OrderItem	dependency	CartStore holds a transient array of OrderItem objects before submission.

4. Sequence Diagram

4.1 Overview

A Sequence Diagram shows how objects interact with each other over time. It illustrates the order of messages passed between system components for a specific scenario. Two critical flows are documented here.

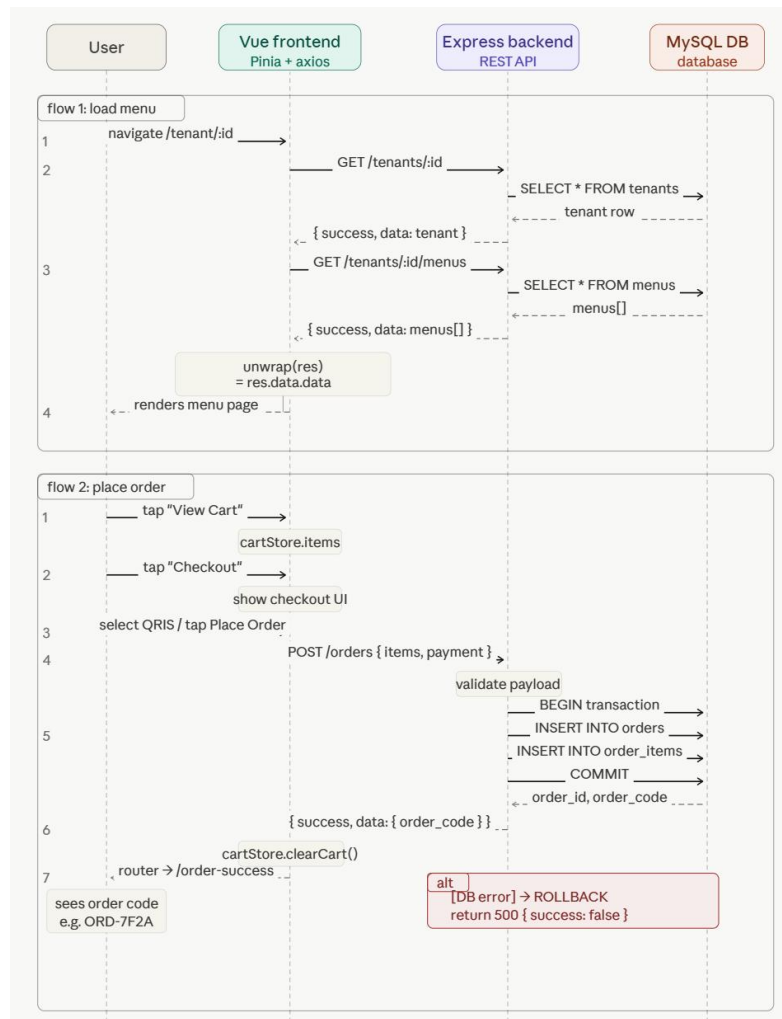


Figure 3.1 – BCanteen Class Diagram

The four lifelines involved in both flows are:

Lifeline	Role
User	The person interacting with the browser or mobile interface.
Vue Frontend	The Vue 3 SPA, including Pinia stores, Vue Router, and Axios HTTP calls.
Express Backend	The Node.js REST API server, responsible for business logic and DB access.
MySQL Database	The relational database storing all persistent data.

4.2 Flow 1: Load Menu

This flow describes what happens when a User navigates to a Tenant Menu page.

Step	From	To	Message / Action
1	User	Vue Frontend	Navigates to /tenant/:id (TenantMenuPage.vue mounts)
2	Vue Frontend	Express Backend	GET /tenants/:id – fetch tenant details
3	Express Backend	MySQL Database	SELECT * FROM tenants WHERE id = :id
4	MySQL Database	Express Backend	Returns tenant row
5	Express Backend	Vue Frontend	{ success: true, data: tenant }
6	Vue Frontend	Express Backend	GET /tenants/:id/menus – fetch menu items
7	Express Backend	MySQL Database	SELECT * FROM menus WHERE tenant_id = :id
8	MySQL Database	Express Backend	Returns menus[]
9	Express Backend	Vue Frontend	{ success: true, data: menus[] }
10	Vue Frontend	Vue Frontend	unwrap(res) = res.data.data – extract payload
11	Vue Frontend	User	Renders tenant hero + category tabs + menu items

4.3 Flow 2: Place Order

This flow describes the complete order submission lifecycle, from the User tapping View Cart through to receiving the order code.

Step	From	To	Message / Action
------	------	----	------------------

1	User	Vue Frontend	Taps “View Cart” button on TenantMenuPage
2	Vue Frontend	Vue Frontend	Reads cartStore.items from Pinia store
3	User	Vue Frontend	Taps “Checkout” from CartPage
4	Vue Frontend	User	Renders CheckoutPage (order summary, payment selection, notes field)
5	User	Vue Frontend	Selects payment method (QRIS or M-Banking), taps Place Order
6	Vue Frontend	Express Backend	POST /orders – body: { items[], payment_method, notes, tenant_id }
7	Express Backend	Express Backend	Validates payload (required fields, item availability)
8	Express Backend	MySQL Database	BEGIN transaction
9	Express Backend	MySQL Database	INSERT INTO orders (tenant_id, total_amount, payment_method, ...)
10	Express Backend	MySQL Database	INSERT INTO order_items for each line item (quantity, unit_price snapshot)
11	Express Backend	MySQL Database	COMMIT
12	MySQL Database	Express Backend	Returns generated order_id and order_code
13	Express Backend	Vue Frontend	{ success: true, data: { order_code: “ORD-7F2A” } }
14	Vue Frontend	Vue Frontend	cartStore.clearCart() – resets cart and tenantId
15	Vue Frontend	User	router.push({ name: ‘order-success’, params: { code } })
16	User	User	Sees order code displayed on OrderSuccessPage

4.4 API Response Convention

All backend endpoints use a consistent success wrapper:

Pattern	Example
Success response	{ success: true, data: <payload> }
Error response	{ success: false, message: "Error description" }
Frontend extraction	unwrap(res) = res?.data?.data ?? res?.data ?? res

The unwrap() utility function (src/utlis/apiHelper.js) handles all three possible structures to make the frontend resilient to API changes.

5. Entity Relationship Diagram (ERD)

5.1 Overview

The Entity Relationship Diagram models the BCanteen MySQL database. It defines all tables, their columns with data types, primary keys, foreign keys, and the cardinality of relationships between tables.

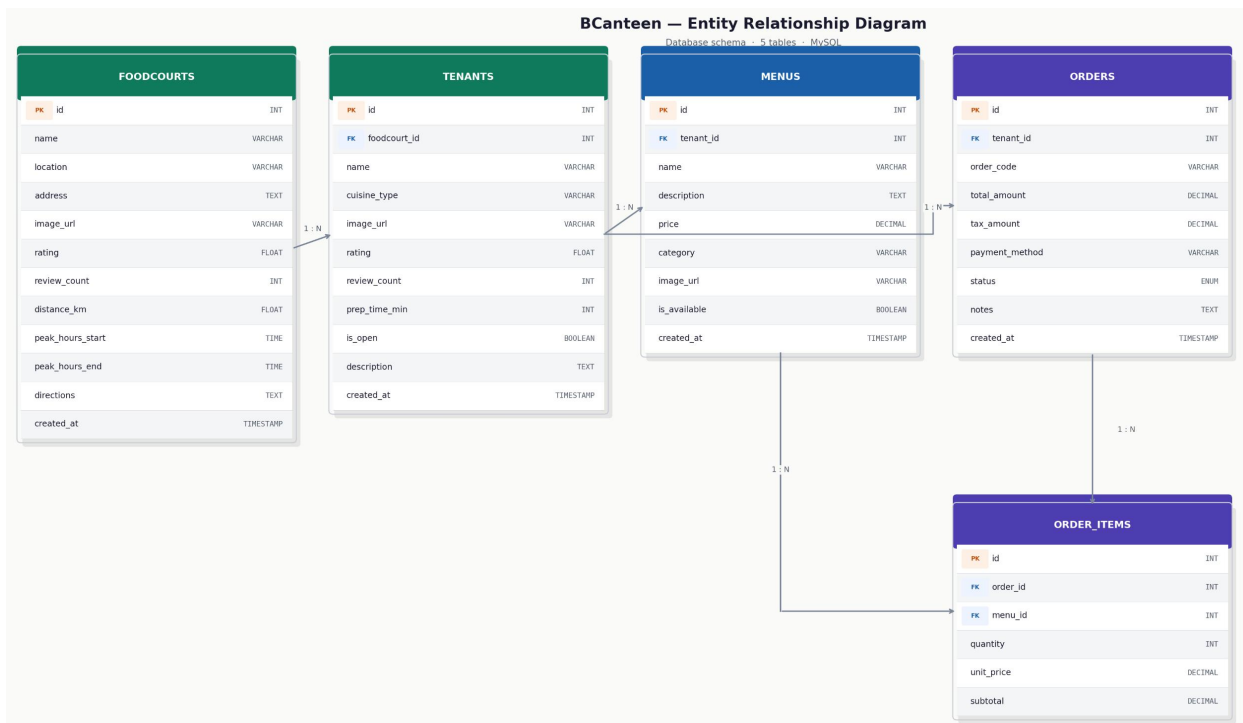


Figure 5.1 – BCanteen Entity Relationship Diagram

5.2 Table Descriptions

foodcourts

Stores the master list of all physical foodcourt locations. It is the root entity of the domain hierarchy.

Column	Type	Constraint	Notes
id	INT	PK, AUTO_INCREMENT	Unique identifier
name	VARCHAR(100)	NOT NULL	Display name
location	VARCHAR(200)	NOT NULL	Short location label
address	TEXT		Full address for directions
image_url	VARCHAR(500)		Hero image URL
rating	FLOAT		Aggregate rating (0.0–5.0)
review_count	INT	DEFAULT 0	Total number of reviews
distance_km	FLOAT		Distance from user (optional)
peak_hours_start	TIME		e.g. 12:00:00
peak_hours_end	TIME		e.g. 14:00:00
directions	TEXT		How to find the foodcourt
created_at	TIMESTAMP	DEFAULT NOW()	Record creation time

tenants

Column	Type	Constraint	Notes
id	INT	PK, AUTO_INCREMENT	Unique identifier
foodcourt_id	INT	FK → foodcourts.id	Parent foodcourt
name	VARCHAR(100)	NOT NULL	Stall name

cuisine_type	VARCHAR(50)		e.g. Japanese, Fast Food
image_url	VARCHAR(500)		Stall logo / banner
rating	FLOAT		Aggregate rating
review_count	INT	DEFAULT 0	
prep_time_min	INT		Average preparation minutes
is_open	BOOLEAN	DEFAULT TRUE	Live open/closed status
description	TEXT		Short description of the stall
created_at	TIMESTAMP	DEFAULT NOW()	

menus

Column	Type	Constraint	Notes
id	INT	PK, AUTO_INCREMENT	
tenant_id	INT	FK → tenants.id	Parent tenant
name	VARCHAR(100)	NOT NULL	Item name
description	TEXT		Displayed below item name
price	DECIMAL(10,2)	NOT NULL	Current selling price
category	VARCHAR(50)		Used for tab grouping (Noodles, Rice...)
image_url	VARCHAR(500)		Food photo
is_available	BOOLEAN	DEFAULT TRUE	Controls sold-out overlay
created_at	TIMESTAMP	DEFAULT NOW()	

orders

Column	Type	Constraint	Notes
id	INT	PK, AUTO_INCREMENT	
tenant_id	INT	FK → tenants.id	All items must be from this tenant
order_code	VARCHAR(20)	NOT NULL, UNIQUE	Human-readable code e.g. ORD-7F2A
total_amount	DECIMAL(10,2)	NOT NULL	Sum of all line items
tax_amount	DECIMAL(10,2)		10% tax stored separately for auditing
payment_method	VARCHAR(20)	NOT NULL	QRIS or M-Banking
status	ENUM	DEFAULT 'pending'	pending confirmed ready completed
notes	TEXT		Customer special instructions
created_at	TIMESTAMP	DEFAULT NOW()	

order_items

Column	Type	Constraint	Notes
id	INT	PK, AUTO_INCREMENT	
order_id	INT	FK → orders.id	Parent order
menu_id	INT	FK → menus.id	Reference to menu item
quantity	INT	NOT NULL	How many units ordered
unit_price	DECIMAL(10,2)	NOT NULL	Price snapshot at time of order
subtotal	DECIMAL(10,2)	NOT NULL	quantity × unit_price

5.3 Relationships

Relationship	Type	Foreign Key	Description
foodcourts → tenants	1 : N	tenants.foodcourt_id	A foodcourt has many tenants.
tenants → menus	1 : N	menus.tenant_id	A tenant offers many menu items.
tenants → orders	1 : N	orders.tenant_id	A tenant receives many orders.
orders → order_items	1 : N (mandatory)	order_items.order_id	Each order has one or more items.
menus → order_items	1 : N	order_items.menu_id	A menu item can appear in many orders.
